Architectural Intent

Aegis Agent is my local-first multi-agent engineering system. Its architecture separates task coordination, specialist reasoning, human authority, tool access and persistence. The objective is controlled intelligence: useful reasoning must not become an implicit authorization to act.

This document describes the public Aegis architecture and the separately implemented private intelligence backend. The two systems are distinct. The private backend has not yet been connected to Aegis, and no integrated runtime behavior is asserted.

The engineering priorities are security by design, minimal necessary context, disciplined token consumption, reproducible reasoning boundaries and maintainable interfaces. Efficiency is achieved by selective context construction, not by weakening authorization or concealing failure.

System Composition and Responsibility

The Aegis host coordinates six named specialists: Alina, Kian, Bita, Aylin, Ahmad and Amin. The inspected public coordination layer includes task-to-agent routing, provider selection, response handling and a multi-specialist workflow. The actual behavior of each path must be understood from its implementation rather than inferred from an agent title.

Specialists produce analyses, plans or proposed operations within the host's policy boundary. The intelligence layer may supply evidence and structured context, but cannot independently approve actions, grant tool permissions or alter the user's authority.

The private backend remains an independent intelligence subsystem. Its responsibility is source understanding, task-scoped evidence selection, context integrity and efficient retrieval. Integration is an interface concern, not a claim that the host currently invokes that subsystem.

Architectural Separation and Trust

The architecture distinguishes the user and approval authority, Aegis orchestration, specialist agents, private context services, source repositories, persistence and external tools. Every transition between these domains is an explicit trust boundary.

Retrieved documents, code comments, repository content and model outputs are untrusted data. They cannot become instructions with higher privilege merely because they are relevant to a task. Permission checks must occur in deterministic application logic outside the model.

A secure integration contract binds each request to the authorized principal, active task, permitted sources, agent role and granted capabilities. A backend response can enrich a decision; it cannot replace a separately required human approval.

Multi-Agent Coordination

The public coordination implementation maps tasks to specialist agents and exposes a workflow that invokes the six specialists in sequence. This is an observable orchestration pattern, not evidence of a fully autonomous distributed planner or of private semantic retrieval in the public code.

Each specialist should receive the smallest evidence set necessary for its responsibility. Agent handoffs should preserve provenance, authorization scope and the distinction between observations, inferences and recommendations.

Multi-agent disagreement is a useful verification signal. Contradictory findings should remain visible until reconciled by evidence or an authorized decision. A shared unrestricted context pool would undermine task isolation and increase unnecessary token consumption.

Private Intelligence Backend

The private intelligence backend is a separately implemented subsystem, not yet attached to the public Aegis execution path. Its described capabilities include AST-driven semantic graphs, task-scoped retrieval, context purity and SQLite concurrency management.

These capabilities are presented as properties of the private system as described by its developer. The private source and runtime have not been independently inspected in this public architectural account. This distinction preserves technical accuracy without exposing proprietary implementation.

An eventual adapter should exchange bounded, typed, authenticated requests and source-attributed responses. The integration must preserve the Aegis approval path, safe failure behavior, explicit version compatibility and a clean separation between intelligence and execution.

AST-Driven Structural Understanding

An abstract syntax tree provides a structural representation of source code. Language-aware extraction can identify modules, definitions, imports, references, function calls and source spans, while distinguishing parsed facts from inferred relationships.

Stable symbol identity, parser coverage, source revision consistency and incremental invalidation are fundamental to a trustworthy structural index. Partial parses and ambiguous references should be labeled rather than silently treated as complete semantic understanding.

Structural retrieval should prioritize the symbols and dependency neighborhoods relevant to a task. This approach reduces irrelevant prompt material and supports precise explanations grounded in source locations.

Semantic Graph and Provenance

A semantic graph connects code entities through typed relationships such as definition, import, reference, invocation and dependency. Each material relationship should retain its source location, revision context and confidence classification.

Graph traversals must respect repository permissions and task scope before producing candidate evidence. Cross-repository edges require explicit authorization. The existence of a graph relation does not, by itself, prove runtime behavior or establish permission to execute code.

Provenance allows an engineering conclusion to be traced to the source material from which it was derived. A source-backed conclusion should distinguish directly observed facts from interpretations, assumptions and unresolved uncertainty.

Task-Scoped Retrieval

Task-scoped retrieval restricts evidence selection to the active authorized request. The retrieval service should validate identity, source permissions, task binding and applicable policy before searching any private index.

Scope must survive caching, reranking and multi-agent handoff. A cache entry generated for one task or authorization class cannot be reused under a broader identity merely because its query text matches.

Results should be deduplicated, ranked for relevance and limited by a deliberate context budget. Narrow retrieval improves privacy, reduces contamination and avoids unnecessary model tokens without sacrificing access to authorized evidence.

Context Purity and Integrity

Context purity means that a model receives evidence appropriate to the current task, with no unauthorized material introduced through retrieval, cache reuse or agent handoff. It is a controlled engineering property, not an unsupported claim of perfect accuracy.

A context envelope can bind task identity, policy scope, ordered evidence, source provenance and a content digest. Once prepared for a model invocation, the evidence snapshot should remain immutable.

Source text is always labeled as data. Instructions embedded in retrieved files must not override system policy, alter tool grants or redefine the user's intent. Output validation and deterministic authorization remain essential beyond prompt-level defenses.

Security Architecture

Security is organized as overlapping controls: verified identity, least-privilege authorization, explicit trust boundaries, input validation, safe secret handling, controlled execution and auditability. No individual safeguard is treated as sufficient.

Operational credentials must not depend on permissive default secrets. Sensitive stored data requires protection appropriate to its classification and threat model. Transport authentication, key rotation, retention policy and safe diagnostics are separate responsibilities.

Prompt injection, poisoned source files, malicious tool output, path traversal, cache-scope confusion and unauthorized cross-task access must be treated as realistic adversarial conditions. Model-generated instructions never confer execution authority.

Human Authority and Action Control

The public Aegis codebase includes mechanisms related to human approval, action integrity and scoped tool access. These controls are central to the distinction between an agent recommending an operation and a system actually performing it.

Consequential actions require an authorization decision bound to the proposed action and its relevant context. Changing the target, parameters or authority context invalidates any approval that was specific to the original proposal.

Private retrieval can inform an action proposal but must never bypass approval, mint unrestricted tool credentials or perform side effects through an intelligence-only interface. Enforcement belongs at each real execution boundary.

Token-Efficient Intelligence

Low token consumption is a design objective pursued through source-aware retrieval, structural graph navigation, deduplication, bounded evidence selection, task-specific context and reuse of appropriately scoped results.

Routing and model selection should match task complexity and trust requirements. Compact evidence should retain enough provenance and surrounding source context to avoid misleading compression.

Efficiency is evaluated against correctness and security. Aggressive truncation that omits a relevant dependency, approval constraint or contradictory source is not a successful optimization. No unmeasured token-saving percentage is asserted.

SQLite Persistence and Concurrency

SQLite can support a local-first backend with disciplined connection ownership, short transactions, bounded write queues and snapshot-consistent reads. Write-ahead logging can improve reader/writer overlap, but SQLite still has a single-writer constraint.

Concurrency correctness requires handling busy or locked states, cancellation, process interruption, migration boundaries and recovery from partial indexing. Retry policies must be bounded and safe for the operation involved.

Graph publication should be atomic so that readers never combine incompatible index revisions. Any claim of universally contention-free writes would require a precise workload definition and independent supporting evidence.

Data Protection and Local-First Operation

Local-first operation reduces unnecessary dependence on remote services but does not automatically guarantee confidentiality. Filesystem permissions, secret storage, database protection, log hygiene and optional model-provider egress remain relevant.

Source repositories, prompts, context snapshots and agent receipts may carry different sensitivity levels. Retention, deletion, encryption and redaction should follow the classification of each data category.

The private backend should expose only the minimum evidence required by an authorized Aegis task. Proprietary graph internals, credentials and unrelated private source material do not belong in the public portfolio.

Integration Contract and Compatibility

The future connection between Aegis and its private intelligence backend should use a narrow, versioned adapter. Request contracts identify task intent and allowed source scope; responses provide structured evidence, provenance and explicit failure states.

Authentication and authorization must be verified at the receiving boundary. A client-supplied tenant or task label is not sufficient evidence of authority. Timeouts, cancellation, size limits and integrity checks are required.

Compatibility must be established against the actual source revision selected for integration. A conceptual interface does not prove that an adapter is already installed or that end-to-end behavior has been exercised.

Resilience and Safe Degradation

An unavailable intelligence service must not cause the host to silently broaden retrieval scope or bypass approval. Aegis may continue in a safe reduced-capability mode when the requested operation does not require private evidence.

Authorization failure should deny access. Index inconsistency should prevent unsupported conclusions. Storage pressure should trigger bounded backpressure rather than uncontrolled parallel writes or unbounded queues.

Recovery procedures should preserve the last coherent index, validate policy and schema compatibility, and restore service without leaking private context through exception messages or diagnostics.

Engineering Verification

Verification separates architectural intent, source-observed mechanisms and demonstrated runtime behavior. Source inspection can establish that a control exists in code; it does not prove that every reachable execution path enforces that control.

Security review should exercise denied access, revoked scope, malicious source content, prompt injection, stale context, unauthorized tool requests, concurrency faults and failure recovery. Tests should cover both normal and adversarial paths.

Private implementation can be evaluated through authorized interface-level inspection without publishing proprietary code. Claims in this architecture remain bounded by what is actually observed or disclosed; no unexecuted test is described as passed.

Assurance and Disclosure Boundaries

This architecture distinguishes the publicly inspectable Aegis host from the separately implemented private backend. The latter is not represented as an already integrated feature of the public host.

Professional engineering disclosure does not require revealing internal source code, credentials, graph data or sensitive configurations. It does require precision about component responsibility, trust boundaries, assumptions and limits of verification.

The system is described without artificial claims of absolute security, universal concurrency guarantees or measured performance outcomes. Technical credibility depends on accurate language and explicit boundaries, not superlatives.

Engineering Position

Aegis Agent combines a local-first multi-agent host with a distinct private intelligence architecture designed for structural source understanding, controlled evidence retrieval and task-specific context.

The architecture places human authority above agent autonomy, authorization above relevance, source provenance above unsupported assertion and efficient context construction above indiscriminate token consumption.

The private backend remains separate from Aegis until an authorized integration is implemented and validated. This separation preserves a clear, technically defensible account of the system as it stands.